

School of Computer Science and Engineering, UNSW



Never Stand Still

WK04-01: Roles Blockchain can Play

Software Architecture for Blockchain Applications,
COMP6452

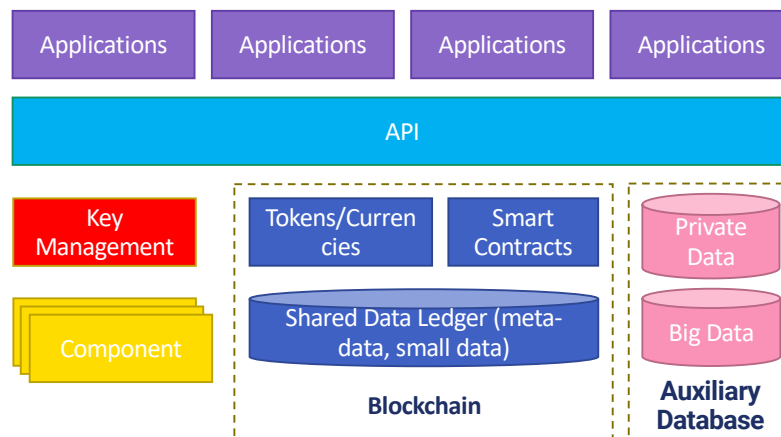
Salil Kanhere, Nadeem Ahmed

Agenda

- Roles Blockchain can play in an architecture
- Storage element
- Computation element
- Communication mechanism
- Asset management & control mechanism

- We'll start with an overview of what roles blockchain can play in a software architecture.
- Those roles include acting as a storage or computational element.
- Blockchains can also enable communication between a set of parties.
- Finally, we see how it can help us manage fungible and nonfungible assets.

Blockchain in a larger Software System



WK 04-01

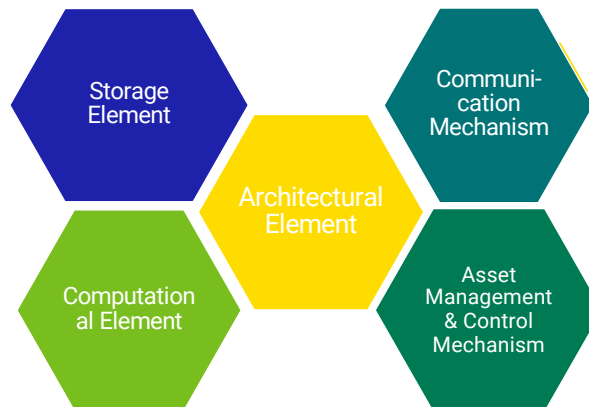
3



- We have seen this diagram before. It depicts how a blockchain may fit in within an architecture of a large software system. This doesn't mean all systems using a blockchain will look like this.
- This is a layered design where applications are at the top and storage is at the bottom. API interconnects the applications with the storage.
 - There's a bit of abasement of the layering where key management as key management and components may be cross-cutting across layers or even sits on top of storage components. Such depictions are common to keep the diagram simple.
- The storage layer is split into 2 main components, blockchain and database.
 - The blockchain stores and shares data and executes smart contracts. The blockchain component might also control digital currency or represent other assets.
 - Due to limitations of privacy and scalability, there are also off-chain auxiliary databases used in the system.
 - First, private data is stored in an internal database. Second, large data is stored separately, such as in a cloud service.
- Key management is an essential component when working with blockchains. Every participant in a blockchain network has one or more private keys, which are used by the participant to digitally sign its transactions.

Roles Blockchain can Play in an Architecture

Blockchains are architectural design choices



WK 04-01

4



- To use a blockchain in the software design process, we need to understand what it can and can't do.
 - These can feed into design options as well as architectural constraints.
- Blockchains can provide 4 key functions as an architectural element in software architecture.
- Within an architecture of a software solution, a BC can be integrated as an element that provides storage and computation.
 - Storage elements – essentially as a database.
 - Computational element – As a computing engine that can run SCs.
- It could also be used as a communication mechanism for multi-party interactions – one party writes data into the BC that the other party can see, e.g., in supply chains.
- Its storage and computation aspects can be combined to provide asset management and control mechanisms too.

Properties of Blockchain

Blockchain is a complex, network-based software component

Properties	Limitations
Immutability for committed TXs	Data privacy
Transparency of TXs & ledger state	Latency for TX inclusion & finality
Integrity for on-chain data	Throughput of TXs???
Non-repudiation that TX happened	Scalability, e.g., size of data on blockchain
Availability for reads	Availability for writes???
Authenticity for digital assets	Authenticity for physical assets

WK 04-01

5



- From the software architecture perspective, blockchain is a network-based software component (more like a distributed database) with unique properties and limitations.
- It has a set of desirable properties and limitations.
- Desirable properties:
 - The most prominent property of blockchain is its immutability where once a transaction is included in the block and then that block is finalized, we can consider that transaction to be committed.
 - We had transparency for both the ledger state and the transactions.
 - Once data are recorded on the blockchain, we can guarantee its integrity. However, not the integrity of data fed into a blockchain. Don't confuse on-chain integrity with off-chain integrity.
 - Once a transaction is recorded on the blockchain its sender cannot repudiate that he/she did not send it. Of course, the sender's key may have been compromised. But at least we know it came from that key.
 - Because blockchain has many nodes you can read data from any of them. Therefore, availability is very high for data reads.
 - As digital assets are managed entirely on-chain we can preserve their authenticity due to properties like integrity and consistency.
- Blockchains also have a bunch of limitations when we compare the technologies like databases.
 - For example, a high level of transparency means we have data privacy problems.
 - As you may have already realized from labs, it takes a while to include transactions in a block and make sure that block is final.
 - Throughput is the number of transactions processed within a unit of time. It is quite low compared to databases where Bitcoin include only 7-8 T transactions per second. For Ethereum is this around 20 TXs/sec? Whereas Visa is known to handle more than 6,000 TXs/sec.
 - Scalability means being able to retain the performance of the system with increasing workload. So, when we talk about workload in blockchains it can include the size of data, number of transactions, number of parties transacting, etc. All of these numbers are somewhat limited in blockchains.
 - While we have so many nodes to read data, we can write data only by following the consensus process. While this means a very high delay, sometimes TXs can get dropped while waiting. Therefore, if we look at this from an availability angle, we can say availability for writes is low.
 - Why a blockchain can protect the authenticity of digital assets does not mean that their physical counterpart is protected similarly. E.g., you may have 10,000 tons of aluminium sitting in the warehouse, and your token says so on the ledger too. But someone can steal that aluminium without changing the token ownership.

Following are formal definitions of some of the quality attributes mentioned above (we'll revisit some of these in a later topic):

- **Integrity** - The degree to which a system, product, or component prevents unauthorized access to, or modification of, computer programs or data
- **Confidentiality** - The degree to which a product or system ensures that data are accessible only to those authorized to have access
- **Non-repudiation** - The degree to which actions or events can be proven to have taken place so that the events or actions can't be repudiated later
- **Authenticity** - The degree to which the identity of a subject or resource can be proved to be the one claimed
- **Availability** - The degree to which a system is operational & accessible when required for use
- **Latency** - How quickly does the system respond to a request?
- **Throughput** - How many requests can the system process in a unit of time?

Agenda

- Roles Blockchain can play in an architecture
- Storage element
- Computation element
- Communication mechanism
- Asset management & control mechanism

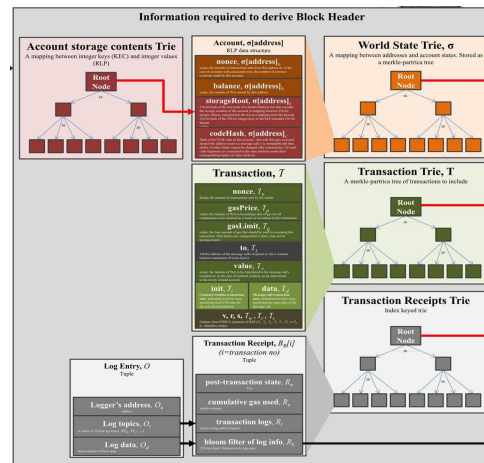
Blockchain as a Storage Element

Append only data store

- Prevents data tampering

Stores

1. Account balances
 2. SC code & state
 3. TXs
 4. Results of TX execution
 - TX receipts & log events
- These provide means to store data on a blockchain



Source: <https://ethereum.stackexchange.com/Questions/268/ethereum-block-architecture>

- Blockchain is an append-only data store that prevents data from being tampered. So you write more and more data, but you don't lose historical data.
- Data can be stored on a blockchain in several ways:
 - Public blockchain provides some form of keeping track of account balances, i.e., who owns how much. This tracks only the cryptocurrency balances.
 - Then we have the smart contract code and the state of those smart contracts. This provides rich data storage in terms of the types of data we can store.
 - We can also include data on a transaction just to keep a record of them, as transactions are immutably recorded.
 - When we execute a transaction, we also keep track of its outputs in the form of what is called the transaction receipt. We can also generate a set of notifications called log events while executing a contract. These also provide a means to store some data. This is usually a low-cost way to keep track of data, but they are not as flexible as stated on a smart contract.
- While these approaches, provide the means to store data on the blockchain not all blockchain support all 4, e.g., Bitcoin only supports the first one.
- This figure depicts how Ethereum keeps track of these 4 types of data storage
 - A Trie is a specific form of a Markle tree.
 - So we have Tries to keep track of account balances, the latest ledger state, transactions, and the transaction receipt including the log entries.

Ledger State Representations

UTXOs	Account-balances
Track individual TXs	Track balances/states of individual addresses
No global state, but can be found by aggregating all UTXOs	Maintain current global ledger state
TX consume 1+ UTXO & produce 1+ UTXO	TXs update latest state of balances/variables Can query historical state at a given block
Analogy to money	Analogy to banking
Better privacy	Low privacy
Enable concurrent processing	No concurrent TXs from same address
Fragmentation of UTXOs	Maintained as state, simple to lookup

WK 04-01

8



- As we discussed earlier there are two ways to represent the ledger state.
- UTXO track individual transactions while the account-balance model tracked the balance of individual addresses.
- With UTXO we don't have a global state. But if you want we can collect and aggregate all the UTXOs to find the total. Whereas the account-balance model maintains the current global state of the ledger.
- A transaction consumes one or more UTXOs and produces one or more UTXOs. Whereas in the account-balance model, transactions update the latest state on the ledger. However, the old state is not lost. If you want to find out the state at a certain block number you can issue a query say give me the state of this account or smart contract state at this block number.
- In terms of analogy to real-world things, UTXOs represent how we deal with money. When we make a payment, we use multiple coins and notes, and then we get the balance back, while the merchant keeps the payment amount. The account-balance model is like how banks keep track of money. You will find that the account-balance model is more understandable when it comes to technical implementation.
- Because UTXOs are specific to individual transactions, they provide relatively better privacy as one can't immediately find out the total balance of an account. Also tracking UTXOs is a bit complicated. Whereas if you go to a certain account in Ethereum, you can see the total balance of that account. Thus, we consider this approach to be low in privacy.
- Because UTXOs are specific to individual transactions we can process multiple UTXOs belonging to the same user in parallel (the same UTXO can't be used in different transactions though). However, in the account-balance model, 2 different transactions from the same account cannot happen because if one transaction updates the balance that should be considered in the other parallel transaction.
- The key problem with UTXOs is the fragmentation where let's say if you have 100 BTC you can issue 100,00 UTXOs to represent them. Whereas in Ethereum 10,000 ETH is still a single value.
- Other benefits of the UTXO Model are:
 - Scalability — Since it is possible to process multiple UTXOs at the same time, it enables parallel transactions and encourages scalability innovation.
 - Privacy — Even Bitcoin is not a completely anonymous system, but UTXO provides a higher level of privacy, as long as the users use new addresses for each transaction. If there is a need for enhanced privacy, more complex schemes, such as ring signatures, can be considered.
- The benefits of the Account/Balance Model are:
 - Simplicity — Ethereum opted for a more intuitive model for the benefit of developers of complex smart contracts, especially those that require state information or involve multiple parties. An example is a smart contract that keeps track of states to perform different tasks based on them. UTXO's stateless model would force transactions to include state information, and this unnecessarily complicates the design of the contracts.
 - Efficiency — In addition to simplicity, the Account/Balance Model is more efficient, as each transaction only needs to validate that the sending account has enough balance to pay for the transaction.

Blockchains vs Other Data Storage Options

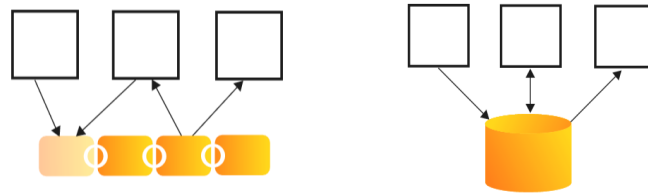
Databases

- Centralised & distributed

Cloud storage

Peer-to-Peer storage

Replicated state machines



- Let's compare how blockchain stacks up against other data storage options.
- While it is not an apple-to-apple comparison, we should consider other data storage options like databases (e.g., Oracle), cloud storage (e.g., AWS S3), peer-to-peer storage (e.g., IPFS), and replicated state machines (e.g., Paxos).
 - Cloud storage includes databases hosted on the cloud. But here we are referring to bulk (object) storage like AWS S3.

Blockchains vs Databases

	Blockchains	Centralised Databases	Distributed DBs (Master-Slave)	Distributed DBs (Master-Master)
Storage	Replicated	None	Replicated	Replicated & Shards
Behaviour	Like a log Based on consensus	CRUD operations	CRUD operations on Master Log sent to slave	CRUD operations on any Master Distributed TXs
ACID properties	ACI, but D???	All	All	All
Trust	Decentralised	Centralised	Centralised	Centralised
Read performance	High	Medium	High	High
Write performance	Low	High	High	Medium
Availability	Very high	Low	Medium	High

WK 04-01

10



- Here we compare blockchains with different database configurations, namely centralised databases and distributed databases based on master-slave and master-master model
 - Master-slave – An active master node, and passive multiple slave nodes that only become active when the master node fails.
 - Master-master – more than one active master node.
- Storage – Both blockchains and distributed databases replicate data. However, the level of replication is very high in blockchains as they may have hundreds to 1000s of nodes. Whereas a distributed database is likely to have < 10 nodes.
- Behaviour – A blockchain behaves like a log of transactions where all changes are append-only. What transactions to append is determined based on consensus. Databases support CRUD operations.
 - CRUD - Create/Read/Update/Delete operations to manipulate data.
 - A blockchain is an append-only data store that does not support update operations.
 - On master-slave design, CRUD operations are performed on the master and all changes are sent to the slave as a log. Then slave executes operations on the log one after another.
 - On master-master design, CRUD operations can be performed on any master. However, a complex distributed transaction management solution is needed to ensure all masters are updated consistently.
 - Traditional shared data stores have their consensus protocols to synchronize replicas in a fully trusted environment, such as 2-Phase Commit and Paxos.
- ACID properties – While databases are designed to achieve ACID, blockchains fail on 1 or 2 of these properties. E.g., durability is not achieved when consensus is probabilistic.
 - ACID (atomicity, consistency, isolation, durability) is a set of properties of database transactions intended to guarantee data validity despite errors, power failures, and other mishaps.
 - A – atomicity, C – Consistency, I – Isolation, and D – Durability
- Trust – In blockchains administration of the network is decentralised. Hence, trust in them is also decentralised. Whereas in databases, the administration of the database nodes is under a single party.
- Performance –
 - Read - In any form of data storage data read performance is higher than write performance. By having more nodes, we can read data from any of them. Therefore, a distributed design, blockchain or not has high read performance.
 - Write – Write performance decreases with more nodes as we need to keep them in sync. So, when you have a single database instance, we can just update as fast. Master-slave performance is also high as master doesn't worry about keeping slave updates. It just ships a log to the slave hoping it will continue to update its copy. However, master-master design and blockchains try to keep all replicas in synch and lose write performance in this process, e.g., the overhead of consensus.
- Availability – Increases when we have more nodes. Centralised databases are prone to a single point of failure. Master-slave is better as a slave can become a master when the master is down. However, this role switch introduces a bit of downtime, e.g., typically a couple of seconds.

Blockchains vs Cloud Storage

	Blockchains	Cloud Storage
Trust	Decentralised	Centralised
Management	Users host	Cloud provider
Data ownership	Public BCs – Nobody owns & unregulated	Users owns, but data sovereignty concerns
Data integrity	High (probabilistic)	Low
Performance & scalability	Low	High
Availability	Very high	High
Cost	High	Low

WK 04-01

11



- Trust – blockchain doesn't rely on any trusted single entity to operate the infrastructure. However, cloud providers are trusted by the users.
- Management – A blockchain network consists of users that host data. By deciding to run a specific version of blockchain client software users can influence how data are stored and managed. Cloud providers store data uploaded by users, and provide access to that data. Users are normally not able to influence how the data is stored.
- Data Ownership – No real owner of data in public blockchains. While you know may know who added the data, there's no way to remove that data if the data owner desires. Whereas cloud providers at least in their agreements recognize that users own data. But there could be data sovereignty concerns where data are subject to not only the user's jurisdiction but the cloud providers too.
 - Data sovereignty is the idea that data are subject to the laws and governance structures of the nation where they are collected.
- Data integrity – Blockchains guarantee data integrity (could be probabilistic) through consensus and cross-checking of each other's data. The same level of data integrity may not be guaranteed by a cloud provider. However, cloud providers normally have a clearly defined service level agreement for the users. No such guarantee for blockchains.
- Performance & scalability – as discussed so far blockchains are quite poor when it comes to performance (e.g., time to finality and throughput) compared to cloud storage.
- Availability – Both rely on replication; hence, has good availability. But the number of replicas in a blockchain can be a couple of orders of magnitude higher.
- Cost – the cost of storing data on a blockchain is high, even though it has reduced substantially in recent blockchain platforms. However, it is still high compared to cloud storage cost which is < \$1 per GB per month.

Blockchains vs Peer-to-Peer Data Storage

Users access data stored on others' computers connected via a P2P network

- BitTorrent, IPFS (InterPlanetary File System), Storj

	Blockchain	P2P Data Storage
Trust	Decentralised	Decentralised
Management	Users host & independently verify data	Users host & can independently verify data
Data integrity	High (probabilistic)	Medium
Availability	Very high	Low for unpopular data
Performance & scalability	Low	Medium
Cost	High	Medium

WK 04-01

12



- Peer-to-Peer (P2P) systems use various mechanisms to share data with peers and to replicate data across nodes.
 - E.g., BitTorrent allows access to the same file stored on multiple nodes. This enhances availability and fast downloads. The same idea is used in IPFS which aims to create a global, distributed file system that can provide a more efficient, permanent, and censorship-resistant way to store and retrieve data.
- Trust – is decentralized in both options as data are held by many nodes.
- Management – In both designs, it is the users that host and manage data. However, others could use the service without hosting data.
- Data integrity – Blockchain has better integrity due to immutability and consensus process compared to P2P storage. Given multiple replicas, even if some copies are modified, it may be possible to recover data from other nodes in the P2P network.
- Availability - Blockchain has high read availability due to high replication. In P2P storage, the availability of unpopular data is low, because the data is replicated and distributed only when the user wants to store or distribute the data.
- Performance and scalability - Blockchain is not suitable for storing data with large sizes, peer-to-peer data storage, on the other hand, can store larger data compared with blockchain. However, both experience high latency though blockchain latency may be an order of magnitude higher.
- Cost – P2P storage cost is lower compared to blockchains, but they are not as low as cloud storage. While most P2P solutions today run as an altruistic service, they should not be considered forever free.

Blockchains vs Replicated State Machines

Fault-tolerant services that maintain state across replicated servers

- Same start state, inputs in same order, outputs, & end state

	Blockchains	Replicated State Machines
Replication	Replicated state & TXs	Replicated state & state updates
Fault Tolerance	Very high	High
Consensus	Only 1 among conflicting proposed TXs is included	Ensure only 1 client acquires a lock
Voting	Large portion of nodes must agree	A quorum of voters
Facilitation	Blocks & TXs are totally ordered	TXs are ordered
Performance	Low	Medium

WK 04-01

13



- Replicated State Machines (RSMs) are fault-tolerant services that maintain state across replicated servers
 - Are a group of methods for implementing a fault-tolerant service that is also highly available and consistent (guarantee consistency over availability).
 - They replicate servers and coordinate client interactions with server replicas.
 - The idea is to have multiple copies of a deterministic state machine that start in the same state and receive the same inputs in the same order, producing the same outputs and reaching the same state.
 - Even if some replicas fail, the service can continue to operate correctly with the remaining replicas.
 - Replicated State Machines existed before blockchains now used with various modifications to achieve high performance in DLTs.
- The design of Blockchain is similar to RSMs in the following aspects.
- Replication – is used in both.
- Fault tolerance – The number of nodes in blockchains is 1 to 3 orders of magnitude larger than RSMs. Hence, they have much higher fault tolerance.
- Both use a consensus that guarantees that only one request or transaction among multiple conflicting ones is responded to or approved. However, their implementation aspects are very different, e.g., RSMs use the idea of locking a state to provide exclusive access to data during a transaction.
- Voting - Both use voting mechanisms to achieve consensus when a quorum of voters agrees to the same result.
- Facilitation - an RSM totally orders the requests from components. Blocks in the blockchain data structure are totally ordered, and so are the transactions within a block. RSMs typically don't have a concept of blocks and work on one transaction at a time (TX batching is not uncommon).
- Performance – RSMs tend to have better performance mainly due to the smaller size of the network.

Exercise

What sort of data storage option will you choose for the following requirements?

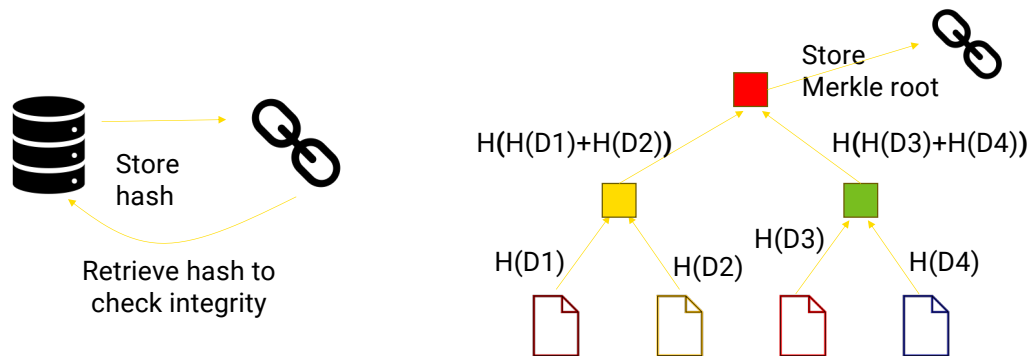
1. High-volume & high-velocity data needing integrity
 - Cloud storage
2. Low-volume & high-velocity data needing integrity
 - Databases (distributed)
3. Low-volume & low-velocity data needing high integrity
 - Blockchain
4. Medium volume & infrequently accessed data needing low-cost data retention option
 - P2P storage

- It's not straightforward to answer these questions and you will see multiple data options can satisfy a given requirement.
 - The given answers are the most obvious options for the scenario (one can argue against these considering other properties not mentioned here).
1. We are talking about big data here. While databases can handle high velocity they are not good at high volume compared to cloud storage. The question is what do we mean by volume, in Bigdata world even a TB is not considered high volume.
 2. These are more like financial transaction data. In contrast to the previous question, low volume is a good reason to use a database. Distributed databases can detect integrity issues but can't resolve them.
 3. Data like maintaining a registry of ownership of an asset, e.g., land title. Blockchains can deal with low volume and velocity. High integrity is key.
 4. These are probably that you want to archive. Infrequently accessed and low cost are good reasons to consider P2P storage. Cloud is also not a bad option (actually more people will go for this over unproven P2P solutions).

Enhancing the Integrity of Off-Chain Data

On-chain data by embedding in TXs, ledger, & event log
Embed a representation of off-chain data as on-chain data

- Typically use data hash
- Can only detect integrity violations, can't recover tampered data



WK 04-01

15



- We can store data on-chain by embedding them in transactions, ledger, and event logs. So they achieve high immutability, integrity, transparency, etc.,
- While we don't get the same properties when data are stored off-chain, we can try to improve the integrity of off-chain by storing a signature of the data on-chain.
 - This is useful for applications that need Proof of existence, proof of integrity, and proof of ownership
 - Some services provide such proofs, e.g., <https://notary.bitcoin.com/>
- We can do this by embedding a representation of off-chain data as on-chain data, i.e., in transactions or ledger.
 - Typically use data hash with transactions. Has is typically 256 to 512 bits. So can fit into a transaction.
 - When we want to check the integrity of off-chain data we can retrieve the hash from the on-chain, and then compare it with the hash calculated from off-chain data.
 - This can only detect integrity violations. We can't recover if data is found to be tampered with.
- We can use a Merkle tree to summarise a large collection of data as illustrated in the figure. E.g., though 4 hashes may be too much to be stored on-chain strong Merkle root reduce the storage requirement by 4 times.

Agenda

- Roles Blockchain can play in an architecture
- Storage element
- Computation element
- Communication mechanism
- Asset management & control mechanism

Blockchain as a Computational Infrastructure

Smart contracts enable on-chain computations

Trusted, transparent, & automated computation, but

- Limited computations, e.g., TX gas limit
- Indeterministic operations not allowed, e.g., floating point & random numbers
- Low performance

On-chain vs. off-chain data determines where computations can be performed

Architectural decision

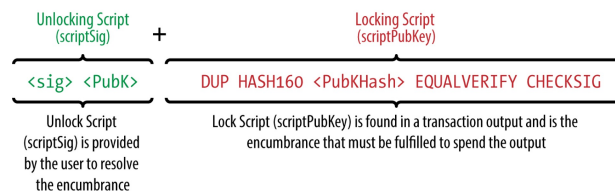
- Which parts of computation should be placed on-chain or kept off-chain?

- Blockchain can be a software component offering computational capabilities.
- This is through smart contracts that allow us to execute small programs on-chain.
 - Smart contracts on Ethereum can express triggers, conditions and business logic, to enable complex programmable behaviours.
- As the same contract code is transparent and deterministic, this enables trust, transparent, and automated (not quite automated as a transaction must always trigger the computation) computation.
 - However, the amount of computations that we can perform is limited, e.g., in Ethereum, it is limited by transaction gas limit. Similarly, there's a block gas limit.
 - To make sure all nodes agree on the output of a smart contract, indeterministic operations such as floating point and random number generation are typically not allowed. Even if allowed (e.g., Hyperledger Fabric), you better not use it
 - Compared to executing the same piece of code on your laptop there is a high overhead in executing the same smart contract code on all nodes in the entire network and latencies are very high as we have to wait till the transaction gets included and confirmed.
- How much computation we can perform also determined where data resides. If data are kept off-chain for reasons such as high volume and privacy, we can't perform many computations on-chain.
- So, our architectural decision should be focused on which parts of computation should be placed on-chain or kept off-chain.

1st Generation – Limited Computational Power

Bitcoin has a simple scripting language that allows storing small amounts of metadata with a UTXO (e.g., 40 bytes)

- Can be used to specify how a TX can spend a UTXO, e.g.,
 - Require 0+ signatures
 - Burn coins
 - Not spendable until a certain time
 - Solve a puzzle, e.g., must submit data that match hash attached UTXO



- While we don't attribute smart contracts with Bitcoin, Bitcoin has a simple scripting language that allows storing a small amounts of metadata with a UTXO (e.g., 40 bytes).
 - The smart contract language on Bitcoin is called Script.
 - We can see a Script instruction below.
 - On the right slide is a simple script attached to a Bitcoin transaction, which is comprised of a locking script defined in a transaction output, which must be fulfilled to spend the transaction output, and an unlocking script provided by the user.
 - To validate the transaction, the miner combines the unlocking script and locking script, and the valid transaction will return true.
 - The unlocking script is that 40 bytes attached to UTXO.
- This can be used to specify how a TX can spend a UTXO, e.g., require 0+ signatures, burn coins, not spendable until a certain time, and solve a puzzle, e.g., must submit data that match the hash attached UTXO.

2nd Generation – Turing–Complete Programming

Turing complete programming language

- As expressive as other general-purpose programming languages
 - E.g., Solidity, JavaScript in Hyperledger Fabric, Java in R3 Corda
- Not always desired as determinism is lost
 - E.g., DAML

Practical limitations on computational complexity

- TX & block gas limits
- Limited memory
- Limited data & querying
- Indeterministic operations not allowed or should not be used

- Ethereum represents the second generation of blockchain, which provides a general computational platform with a Turing complete programming language
 - Such programming language is as expressive as every other general-purpose programming language like Java, C++, and Python.
 - Solidity, JavaScript in Hyperledger Fabric, and Java in R3 Corda are examples of Turing complete smart contract languages.
- While such languages are rich in functionality, they are not always the best as we may lose indeterminism, e.g., DAML does not allow certain operations to make sure DAML outputs are deterministic and analysable.
- While the languages are rich there are some practical limits on what computations we can perform, e.g.,
 - Gas limits apply to both the transaction level and block level. This is done to prevent Denial of Service (DoS) attacks on the network.
 - Amount of memory you have access to is likely to be a few 10s of MBs.
 - Limited data access is another problem, e.g., you can't perform complex queries on ledger data within a smart contract.
 - Indeterministic operations are either explicitly not allowed or you should not use them.

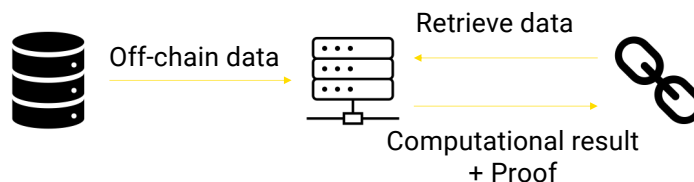
Enhancing Integrity of Off-Chain Computations

Get off-chain components to compute based on on-chain or off-chain data as inputs

- Essentially an oracle

Use CPU enclave to execute trusted code, calculate integrity proofs, & protect data privacy

Zero-knowledge proof, Homomorphic encryption to enhance privacy, trust, & integrity

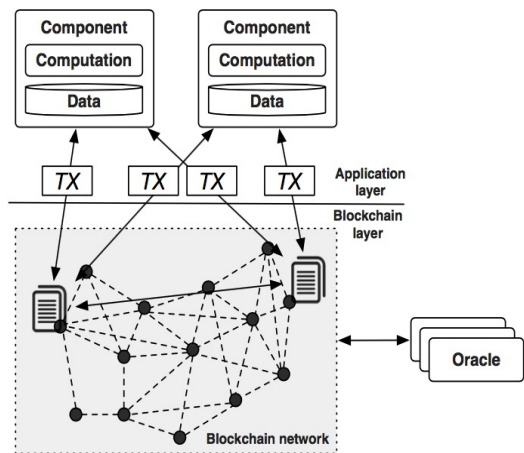


- Given the practical limitations of smart contract execution and the volume of data we can store on-chain, we may need to perform part of a DApp's computation off-chain.
- In that case, we can still use a blockchain to enhance the integrity of off-chain computation.
- We can get an off-chain component to perform the computation based on on-chain or off-chain data as inputs. Then it can submit the results and some integrity proof back to the blockchain to confirm the computation was performed as expected.
 - In that case, the off-chain component is essentially an oracle. So we have benefits and limitations of oracles like trust issues, cost, and single point of failure (if a single oracle).
- We can use CPU enclaves like Intel SGX to execute trusted code, calculate integrity proofs, and protect data on the on-clave.
 - We can leverage Zero-knowledge proof or Homomorphic encryption to enhance the privacy, integrity, and trust in off-chain computation. However, these topics are beyond the class's scope.

Agenda

- Roles Blockchain can play in an architecture
- Storage element
- Computation element
- Communication mechanism
- Asset management & control mechanism

Blockchain as a Communication Mechanism



TXs get broadcast across the whole network

A user can issue a TX so that another user can act on it

- Can see TXs by querying or subscribing to log events
- E.g., noticing *transfer* event in ERC-20 contract, Alice can ship a bicycle to Bob

Need to handle chain forks

Common practice is to cache/dump blockchain data to a database & query it

- Blockchain provides a communication mechanism through transactions.
- In the figure, both components read transactions from other parties and write transactions other parties can read.
- Because transactions are visible to anyone on the network, the transaction receiver or a third party can act based on the announcement of the transaction.
 - The existence or completion of a transaction can be checked by querying a blockchain client.
 - We can also subscribe to log events, e.g., when a token gets transferred in an ERC-20 contract, the contract emits a Transfer(from, to, value) event. Based on this event we can find out the sender, receiver, and transaction value.
 - So, if Alice is expecting a payment in some token (or even Ether), she can wait for the transfer event and then ship the bicycle to Bob.
- However, in this case, Alice takes the risk that the transaction is not included in the main chain and may eventually go back to the transaction pool.
- A common practice is to cache or dump all blockchain data into a database and perform rich queries on it. However, it is nontrivial to keep the database in synch with changes on the blockchain.
- In summary, as a transaction on a blockchain is visible to other (interested) parties, we do not need to explicitly communicate such information off-chain. One party writes data, and it is visible to anyone interested and allowed to see based on access control rules. This is very useful in domains such as supply chains.

Computation Communication

Communication enable coordinated computation

- Bob pays, Alice ship bicycle, & Pat confirm delivery
- On-chain → Off-chain → On-chain control flow

Typical control flow

- Initiated from externally owned accounts
- Transferred among contract accounts

Oracle facilitates component coordination with external state

- Block a TX's progress until oracle completes
- Block an external progress until TX completes
- Delay between external state changes & record those on-chain

- Communication can also enable coordination, e.g., sequencing a set of business activities, e.g., Bob pays, Alice ship, and Pat confirm delivery.
- Typical control flow includes:
 - The control flow of an application is initiated from externally owned accounts and is transferred among contract accounts.
 - One contract can call another contract.
- When the validation of certain transactions depends on an external state, oracles are used to facilitate the coordination. We can have behaviours where:
 - Block a transaction's progress until oracle completes.
 - Block external progress until the transaction completes.
 - However, there could be a considerable delay between external state changes and those changes are recorded on-chain.

Agenda

- Roles Blockchain can play in an architecture
- Storage element
- Computation element
- Communication mechanism
- Asset management & control mechanism

As an Asset Management & Control Mechanism

Cryptocurrency & tokens are virtual assets

Typically, only owner can control them

More complex control models can be implemented, e.g.,

- Multi-signature or threshold voting (1+ account controls an asset)
- Escrow – smart contract code controls an asset
- Atomic swap

Highly customisable through smart contracts

- Some platforms bake these asset management mechanisms into protocol
- E.g., Cardano Native Tokens

- Blockchain provides a management and control mechanism for native assets, e.g., cryptocurrency.
- And we can extend these functionalities using smart contracts and issues tokens are also virtual assets.
- Typically, only the owner can control an asset. However, we can implement more complex control models, e.g.,
 - Multi-signature or threshold voting where 1+ account controls an asset. See the next slide.
 - Escrow – smart contract code controls an asset until payment conditions are satisfied or timeout is reached. See the escrow example from the Smart Contracts lecture.
 - Atomic swap allows 2 assets to be exchanged as one transaction, either exchanging both assets or neither.
- Such asset management and control mechanisms can be highly customizable through smart contracts.
 - Some platforms bake these asset management mechanisms into the protocol, see Cardano Native Tokens allow any token to be issued as set of base operations, see <https://docs.cardano.org/native-tokens/learn/>

Example – Multi-Signature

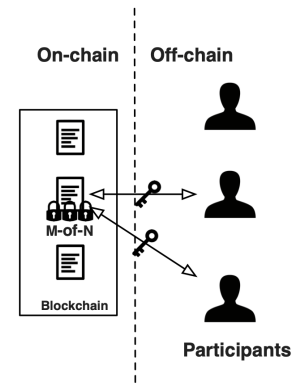
Aka MultiSig or multiple authorisation

Some TXs need to be authorised by multiple parties (joint accounts)

- Some TXs need to be signed by all addresses
- Other require m out of n addresses to sign ($m < n$)
- Some tolerance to lost or compromised keys

Solution

- Predefine m & set of addresses to authorise a TX
 - Bitcoin – Use script hash to specify input conditions for UTXO
 - 2nd generation blockchains – Specify in a smart contract
 - Off-chain – Workflow that splits key among multiple parties
- Get m addresses to sign TX(s)
 - Same TX can be used to specify m & set of addresses for future TXs



Source: <https://research.csiro.au/blockchainpatterns>

WK 04-01

26



- Multi-signature is an example of an asset management and control mechanism.
- In some blockchain-based applications, activities might need to be authorized by multiple blockchain addresses. For example, a monetary transaction of a joint account may require authorisation from multiple blockchain addresses.
 - Sometimes we need all addresses to sign and other times we need a subset to sign.
 - Not all addresses that are authorized to perform an activity might not be able at the time of signing transaction
 - This can also tolerate some-level of lost and compromised keys. For e.g., if a key is lost others can still sign as far as m keys can be found. In case of a compromise, compromised key is insufficient as far as $m > 1$
- Solution
 - It would enable more dynamism if the specific set of blockchain addresses for authorization are not decided before the corresponding transaction being submitted into the blockchain network, or the corresponding smart contract being deployed on blockchain.
 - However, the set of addresses that can be signed need to be predefined
 - On Bitcoin, a multi-signature mechanism can be used to require more than one private key to authorize a transaction. This requires Pay to script hash (P2SH) that allow transactions to be sent to a script hash (address starting with 3) instead of a public key hash (addresses starting with 1). To spend bitcoins sent via P2SH, the recipient must provide a script matching the script hash and data which makes the script evaluate to true.
 - In Ethereum, smart contract can mimic multi-signature mechanism. More flexibly, an M-of-N multi-signature can be used to define that M out of N private keys are required to authorize the transaction.
 - There are also off-chain solutions to do the same off-chain.
 - One approach is for each party to have a private key that can be used to assign and confirm it's ok to sign a transaction with a single key
 - Another is to split the private key among multiple parties, e.g., Fireblocks use a technique called multi-party computing to spit different parts of the key across multiple parties
 - We also need to predefine m
 - Then get those parties to sign the TX(s)
 - Whether this is a single TX or multiple depends on the technique
 - For e.g., in Bitcoin case a single TX can carry multiple signatures
 - For smart contracts you can either send a single TX with all signatures as data or each party can send a separate transaction which the smart contract can keep track to determine whether m out of n is reached
 - In off-chain solutions only a single transaction with single signature is issued. However, to do this that signature need to be generated based on another set of signatures or multi-party computing like in Fireblocks
- m is the threshold of authorization. If $m = n$ all must sign
- m and set of addresses can be updated with time, e.g.,
 - When generating a UTXO input conditions it much satisfy while spending can be specified
 - Smart contract can be designed to update m and set of addresses
- Multi-signatures enable flexible binding of authorities, but depends on the availability of authorities when the activity is proceeded.
- One party can own more than one blockchain address to reduce the risk of losing control over their smart contracts due to a lost private key.

Example – Colored Coins



Identity of a cryptocurrency or associated data can be used to represent other assets

A technique for representing & managing real-world assets on Bitcoin

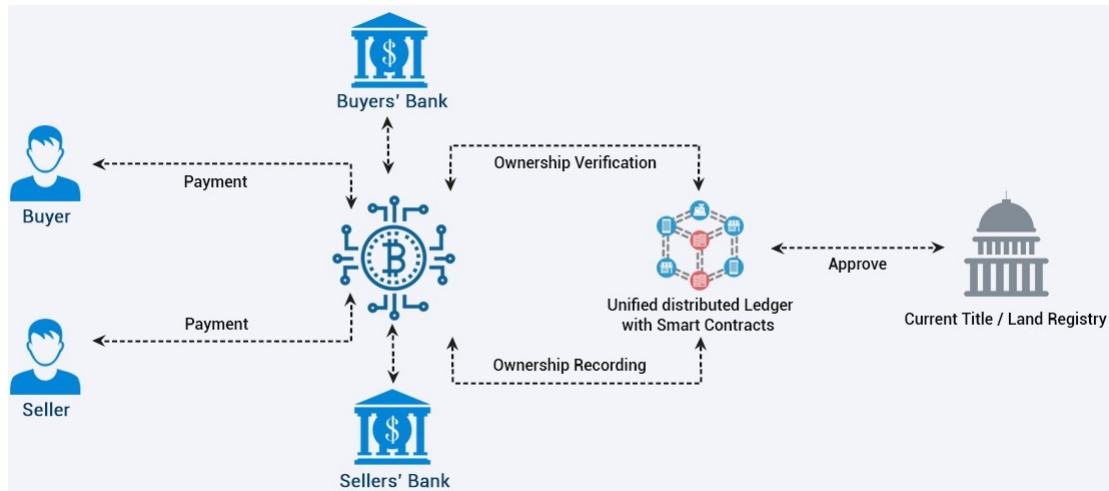
- Limited capabilities due to small size of arbitrary data on scriptSig

A specialised wallet can represent asset manipulation instructions

- E.g., 40-byte metadata on UTXO could represent 100 assets assigned to a Bitcoin address
- A TX can transfer 60 assets from that address to another, while assigning remaining 40 to original owner
- Redemption depends on asset issuer's promise to redeem

- The identity of a cryptocurrency or associated data can be used to represent other assets.
- The term “coloured coin” was created by eToro CEO, Yoni Assia, who co-authored a 2013 whitepaper called “Colored Coins with Ethereum” creator Vitalik Buterin. Assia and Buterin proposed that it's possible to “colour” a set of Bitcoins to distinguish them from the rest.
 - See the “Colored Coins with Ethereum” whitepaper for technical details.
- Coloured coins are essentially a set of techniques for representing and managing real-world assets on Bitcoin.
 - As discussed, Bitcoin has a simple scripting language that allows storing a small amount of metadata with a UTXO (e.g., 40 bytes).
 - This can be leveraged to represent asset manipulation instructions.
 - However, what we can do to manage these coins is limited due to the small size of arbitrary data on scriptSig.
- A specialised wallet can be used to represent asset manipulation instructions, e.g...
 - Metadata on UTXO could represent 100 assets assigned to a bitcoin address.
 - A TX can transfer 60 assets from that address to another while assigning the remaining 40 to the original owner.
 - Redemption depends on the asset issuer's promise to redeem

Exercise – What roles of BC are in operation here?



Source: [Making Blockchain a part of Real Estate Companies only at Analyticsonline.in](https://www.analyticsonline.in/making-blockchain-a-part-of-real-estate-companies-only-at-analyticsonline.in)

We can see blockchain being used as a:

- Storage element, e.g., ownership recording
- Computational element – Approval and verifying ownership. However, these could just be strong approval or verification decisions.
- Communication – Ownership recording may trigger approval. Approval may trigger ownership verification.
- Asset management – Payments. Land title is also an asset here.